

面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v27: 投稿主线压缩版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合把经典谓词嵌入量子算法, 是 Grover 搜索、量子密码分析和约束求解类量子程序中的基础编译步骤。已有方法通常从固定布尔表示、可逆逻辑模板、LUT 映射或 CNOT 导向覆盖结构出发, 但容错量子计算中的逻辑开销同时受 T-count、CNOT、深度、门数和辅助比特影响。本文把该任务重新表述为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源约束搜索问题, 并提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构深度策略、depth-frontier policy 与保守 guard 的 Resource-NMCTS 框架。本文方法不依赖 SSHR 的 parallelotope 候选空间; SSHR 仅作为 small-scale CNOT-oriented baseline 参与比较。

在 $n \leq 6$ 的 177 个传统函数上, Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%; 相对 weighted ESOP-MILP 获得 167/3/7, 平均 score 降低 29.84%。在 ROS-style LUT proxy 中, ABC $K = 3, 4, 5$ sweep 对 $n = 3-6$ 与 $n = 14, 15, 16, 18$ 共 309 个函数产生 927 个映射行, 全部通过 truth-table 检查; best- K proxy 相对固定 $K = 4$ 平均 score 降低 18.12%, 但 Resource-NMCTS 相对该更强 proxy 仍获得 309/0/0, 平均 score 降低 83.77%。RevKit Python API 的 `oracle_synth` 在 $n \leq 6$ 的 177 个函数上全部返回, 但角度审计显示 171/177 行含非 Clifford R_z , 总数为 9242; 因此 RevKit lower-bound phase netlist 不能直接作为精确 Clifford+T T-count 对比。若每个非 Clifford R_z 仅加入 1 个 score 单位, Resource-NMCTS family 相对 RevKit 为 157/20/0; 若采用 Ross-Selinger-style $T/R_z = 30$ 的近似旋转综合 proxy, 则为 177/0/0, 平均 score 降低 95.03%。高维 RevKit 隔离超时探针进一步显示, $n = 14$ 前 8 个函数中仅 1 个在 30 s 内返回、7 个超时, 唯一返回行含 32767 个非 Clifford R_z 。在 $n = 20-40$ 项集级 scale 中, 6600/6600 个方法行通过 ANF plan 展开验证和 emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号验证; 在 $n = 21, 22, 23$ truth-table bridge 中, 300/300 个方法行通过完整 oracle 验证。本文的阶段性结论是: Resource-NMCTS 已形成一条独立于 SSHR、以低 T-count 与低加权逻辑资源为目标的 Boolean oracle synthesis 主线; 但其主张应限定在逻辑层 bit-flip oracle synthesis, 投稿前仍需补强 phase/ R_z -aware emitter、官方 ROS/mockturtle 或 legacy RevKit/CirKit 对照, 以及真实后端相关评估。

关键词: 量子 oracle 综合; 布尔函数综合; 强化学习; 蒙特卡洛树搜索; ANF; FPRM; 资源约束编译

1 研究定位

本文的核心主题是基于强化学习与蒙特卡洛树搜索的量子布尔函数综合方法。更具体地说, 本文把 Boolean oracle synthesis 看作一个可验证的组合搜索问题: 给定布尔函数的 ANF 或 FPRM

项集合，算法需要在大量可计算、可反算、可递归验证的代数分解动作中选择一条低资源线路生成路径。神经网络和 MCTS 的角色不是直接生成量子门序列，而是在候选动作爆炸时更早找到有资源收益的分解组合。

这个定位决定了本文不能从 SSHR 入手。SSHR 的 parallelootope 结构是 small-scale CNOT-oriented synthesis 的重要代表，适合成为 baseline，但它既不是本文方法的内部结构，也不是本文的唯一比较目标。本文更自然的论文主线是：用 ANF/FPRM 项集合定义搜索状态，用 Boolean-ring screen 扩展可用因子结构，用神经先验和 MCTS 处理动作组合，用 depth policy、frontier policy 和 guard 控制预算与风险，最后用 T-count、CNOT、深度、门数、辅助比特和综合 score 评价结果。

一句话概括本文论证：

在逻辑层量子布尔函数 oracle 综合中，将 ANF/FPRM 项集合作为搜索状态，并用资源感知神经搜索选择可计算、可反算的代数分解动作，可以在传统函数和高维项集合上降低 T-count 与加权逻辑资源；其边界是当前结果仍停留在逻辑层，不包含 routing、magic-state factory、真实噪声模型或硬件后端映射。

表 1: 本文术语表与固定写法。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架，包括神经动作先验、MCTS、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支，允许线性因子与 quotient 共享变量。
Depth policy	在 single、depth-1 和 depth-2 screen 之间选择的结构级策略。
Depth-frontier policy	在 depth-2、depth-3 和 depth-4 screen 之间选择的高预算质量-时间折中策略。
Guard	以确定性 baseline 为兜底的保守门控，只有当学习候选不劣于 baseline 时才接受。
Truth-table bridge	在 $n = 21, 22, 23$ 构造完整 truth-table，把大规模项集符号验证和函数级完整验证连接起来。
SSHR	CNOT-oriented small-scale baseline；本文方法不使用 SSHR parallelootope 候选。

2 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，量子 bit-flip oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这一步在量子算法中常被看作“把经典逻辑接进量子程序”，但在容错量子计算中，它可能成为非 Clifford 资源、辅助比特峰值和线路深度的主要来源。若直接按 ANF 单项式逐项实现，语义透明但会重复计算相似结构；若只优化 CNOT，又可能把成本转移到 T-count、辅助比特或深度上。因此，本文关注的问题不是某一单项指标最优，而是多资源约束下的 Boolean oracle synthesis。

本文采用 ANF 作为语义基准：

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

其中 $\mathcal{M}$ 是 square-free monomial 集合。该表示有两个直接优势。第一，任意候选线路都可以回到 GF(2) 多项式进行等价验证。第二，搜索动作可以自然定义为对项集合的代数变换，例如公共因子提取、FPRM 极性重写、线性奇偶因子筛选和递归子问题分解。困难在于，原始 ANF 不显式给出共享结构，动作空间随项数和变量数迅速增长。本文的核心思路是用神经先验和 MCTS 控制这种动作爆炸，同时保留严格的代数验证路径。

本文贡献如下：

- 提出独立于 SSHR 的 Resource-NMCTS 搜索表述，以 ANF/FPRM 项集合为状态，以逻辑层资源 score 为目标，把 Boolean oracle synthesis 写成可验证的组合搜索问题。
- 引入 Boolean-ring screen，在 square-free Boolean ring 中允许线性因子与 quotient 共享变量，从而扩大高维可搜索因子结构。
- 将 AI 模块拆分为动作排序、结构深度选择、depth-frontier 选择和 baseline-preserving guard，避免把学习策略写成不可控的黑箱生成器。
- 在 $n \leq 6$ 传统函数、 $n = 18, 20$ 函数级边界、 $n = 20$ 到 40 项集级 scale、 $n = 21, 22, 23$ 完整 truth-table bridge、ROS-style LUT proxy 和 RevKit API baseline 上给出逻辑层资源、正确性和边界证据。

3 相关工作

3.1 布尔表示驱动的 oracle 综合

Xor-And-Inverter Graphs (XAG) 把 XOR 和 AND 结构分开，使 AND 节点数量与低 T-count oracle 编译直接相关。Meuli 等人讨论了 multiplicative complexity 在低 T-count oracle 编译中的作用 [1]，并将 XAG 用于量子编译 [2]。ROS 将 oracle synthesis 表述为 resource-constrained LUT mapping [3]，进一步说明布尔表示选择会影响 T-count、辅助比特和线路深度。BDD-based reversible synthesis 展示了面向大函数的可扩展布尔表示路径 [4]。后端感知 oracle synthesis 则开始把逻辑综合和 fault-tolerant back-end 指标连接起来 [5]。

本文与上述工作的共同点是承认布尔中间表示决定逻辑资源；差异在于本文不固定使用 XAG、LUT 或 BDD，而是在 ANF/FPRM 项集合上显式搜索可反算的因子结构。这样做的优点是语义验证直接，代价是候选动作空间更大，需要 AI 和 guard 控制搜索。

SSHR 利用 parallelotope 的空间结构做 small-scale Boolean function 的 CNOT-oriented synthesis [6]。本文不使用 SSHR 的候选空间，也不把论文目标设为 CNOT-only 最优。SSHR 在本文中是一个定位明确的 baseline：它帮助说明本文在低 T-count 和加权 score 上的优势，也提醒我们在 CNOT 指标上必须诚实报告 trade-off。

3.2 学习式搜索与线路优化

学习式搜索已经出现在经典布尔电路设计、量子线路综合和 ZX-calculus 优化中。ShortCircuit 使用 AlphaZero 风格搜索进行经典布尔电路设计 [7]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis [8]; 强化学习和图神经网络也被用于 ZX-calculus rewrite-rule 选择 [9]; MonteQ 使用 MCTS 处理量子线路综合中的动作排序问题 [10]。这些工作证明, 离散线路综合中的动作选择可以受益于学习策略。

本文的区别在于状态不是任意量子门序列, 也不是 ZX 图, 而是 Boolean oracle 的 ANF/FPRM 项集合。神经模型不直接输出线路, 而是排序或选择候选代数分解动作; 最终线路仍由可验证的 compute/action/uncompute 结构生成。这一设计降低了 AI 生成错误线路的风险, 也使每个候选都可以经过 ANF 展开、GF(2) 符号模拟或完整 truth-table 检查。

4 问题定义与资源模型

本文输入是布尔函数的 truth-table 或 ANF 项集合, 输出是实现式 (1) 的逻辑层可逆线路。线路由 X、CNOT 和多控 Toffoli (MCT) 抽象门组成, 并在资源统计中估计 T-count、CNOT、depth、gate count 和 peak ancilla。搜索排序使用统一分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数不是物理设备的运行时间或错误率模型, 而是逻辑层候选排序目标。为避免单一分数掩盖 trade-off, 实验同时报告各个资源分量。

正确性验证分三层。第一, 在函数级小规模 and 边界切片中构造完整 truth-table, 逐点检查 oracle 输出。第二, 在大规模项集实验中, 将 factor plan 展开回 ANF 项集合, 检查目标多项式是否等价。第三, 对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟, 检查输出线多项式、输入线复原和辅助线清零。第三层验证不等同于 2^n 个输入的完整枚举, 但它检查的是最终线路而不只是搜索 plan。

5 方法

5.1 搜索状态、动作与线路生成

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构, 将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题, 再递归生成 compute/action/uncompute 形式的线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子, 以及由神经模型扩展的 root actions。

动作 a 被接受后, 框架会估计其即时资源收益、子问题大小、辅助比特峰值和后续可分解性。神经动作先验对候选排序, MCTS 在有限预算内探索候选组合, rollout 使用确定性资源估计和已有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选取最优者。

5.2 布尔环线性因子

传统线性因子筛选容易要求线性因子变量和 quotient 变量不相交。这个限制便于实现，但会排除 square-free Boolean ring 中合法的重叠结构。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (4)$$

线路层面仍可用 CNOT 计算线性奇偶量，再以该辅助量控制 quotient 子线路。这样，Boolean-ring screen 不是简单增加 beam 宽度，而是把原先被实现约束排除的代数候选重新放回可综合空间。

5.3 神经策略、结构策略与保守 guard

本文把 AI 的角色拆成三类。第一类是动作排序：模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择：depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 Boolean-ring screen。第三类是 frontier 选择：depth-frontier policy 在 depth-2、depth-3 和 depth-4 Boolean-ring screen 之间选择。质量型 frontier policy 用 score-only oracle frontier 训练，目标是在接近 depth-4 或 all-depth frontier 的质量时减少全深度枚举开销；cost-aware frontier policy 则在标签中加入相对运行时间惩罚，用较小 score 牺牲换取更短 plan time 和更低辅助线生命周期。

保守 guard 用来限制学习策略的风险。若学习候选相对确定性 baseline 出现 score 退化，则返回 baseline。Depth-2 skip guard 优先避免把应该运行 depth-2 的项集合错误跳过。Screen-gate 则用于判断在某些边界函数上是否可以跳过昂贵的 Resource tail。这样的设计降低了策略模型分布外失误对资源结果的影响，但也意味着部分 AI 贡献表现为运行时间节省或质量-时间折中，而不是无条件的质量支配。

表 2: Resource-NMCTS 的模块、机制与证据钩子。

模块	机制	主要证据类型
ANF/FPRM 搜索状态	把 oracle 综合写成项集合分解问题，所有候选可回到 GF(2) 多项式验证。	小规模 truth-table 验证与项集级 ANF 验证。
Boolean-ring screen	搜索允许共享变量的线性因子，递归处理 quotient/rest 子问题。	$n = 18, 20$ 函数级边界和 $n = 20-40$ scale。
神经动作先验与 MCTS	对候选动作排序并在有限预算内组合分解动作。	传统函数上的搜索消融和 portfolio 改善。
Depth policy / guard	学习 screen 深度或跳过较深搜索，并用 baseline-preserving 规则兜底。	held-out 项集时间节省和无 score-loss guard。
Depth-frontier policy	学习高预算 depth-2/3/4 质量前沿，并提供 score-only 与 cost-aware 两种运行模式。	$n = 20, 28, 40$ scale、独立 seed 泛化与 $n = 21, 22, 23$ bridge。

6 实验设计

实验分为六类。第一类是 $n \leq 6$ 传统函数, 用 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、ABC/BDD 估计、ROS-style LUT proxy、RevKit Python `oracle_synth` 和 SSHR-H/SSHR-I 作为 baseline。第二类是 $n = 14$ RevKit API 隔离超时探针, 对高维 truth-table 输入逐行用子进程硬超时运行。第三类是 $n = 14, 15, 16, 18$ 的高维外部导出集, 对 ABC-AIG/XAG/LUT/BDD 和 ROS-style LUT proxy 做统一 truth-table 校验。第四类是 $n = 18, 20$ 函数级边界, 完整构造 truth-table 并验证 emitted oracle circuit。第五类是 $n = 20$ 到 40 项集级 scale, 不构造完整 truth-table, 而是对 plan 和 emitted circuit 做符号等价验证。第六类是 $n = 21, 22, 23$ truth-table bridge, 对一小组生成式 ANF 函数重新构造完整 truth-table, 专门检查项集级符号验证是否与函数级 oracle 验证一致。

所有 paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行保留为运行边界证据, 但不参与正确结果均值比较。所有结论限定在逻辑层资源, 不声称硬件 mapping、连通性约束或噪声模型下最优。

表 3: 实验层次与每一层回答的问题。

实验层次	数据与对照	回答的问题
传统函数	$n \leq 6$, 177 个函数; ESOP、SSHR、ABC/BDD、RevKit API。	小规模完整 truth-table 条件下是否形成低 T-count 与低 score 优势, 以及 RevKit phase 口径暴露出什么边界。
ROS-style LUT proxy	ABC $K = 3, 4, 5$ sweep, 309 个函数, 927 行检查。	更强 LUT proxy 是否削弱本文方法优势。
高维函数边界	$n = 18, 20$ 函数级切片。	高维完整 truth-table 条件下哪些搜索分支仍可运行。
项集级 scale	$n = 20-40$, 6600 个方法行。	结构策略是否可扩展, 以及 plan 和 emitted circuit 是否符号等价。
Truth-table bridge	$n = 21, 22, 23$, 300 个方法行完整验证。	项集级符号验证和函数级 oracle 验证是否一致。
Schedule proxy	emitted circuit 逻辑层调度。	score 改善是否也反映到 T-depth proxy 与辅助线生命周期。

7 结果

7.1 传统函数上的低 T-count 与低 score 优势

表 4 给出 $n \leq 6$ 传统函数上的平均资源。相对 direct ANF, Pareto-Resource-NMCTS 平均 T-count 降低 72.25%, 平均 score 降低 67.80%。相对 ESOP cube beam, Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。SSHR-H 的平均 CNOT 最低, 因此本文不能主张 CNOT-only 支配; 更准确的结论是, 本文用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低加权 score。

表 4: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

7.2 外部 LUT proxy 与 RevKit phase baseline

官方 ROS 把 oracle synthesis 写成 resource-constrained LUT mapping。当前环境尚不能直接运行官方 ROS/mockturtle 或 legacy RevKit/CirKit 流程，因此本文新增一个明确标注的 ROS-style LUT proxy：对同一导出 BLIF benchmark 运行 `ABC if -K`，在 $K = 3, 4, 5$ 之间做 sweep；每个映射后的 LUT 网络按 local ANF compute/action/uncompute 方式估计 X/CNOT/MCT 逻辑资源，再按式 (3) 选择 best- K 。该 proxy 覆盖 $n = 3-6$ 的 177 个传统函数，以及 $n = 14, 15, 16, 18$ 的 132 个高维导出函数，共 309 个函数。三档 K sweep 产生 927 个外部映射行，全部通过 truth-table 检查，没有 timeout、error 或 incorrect 行。Best- K proxy 相对固定 $K = 4$ 自身获得 219/0/90 的 score 胜/负/平，平均 score 降低 18.12%；因此它确实是比原 ABC-LUT 更强的 LUT baseline。在这个更强 baseline 上，Resource-NMCTS 仍在 309/309 个函数上取得更低 score，平均 score 降低 83.77%。

RevKit API 给出了另一种必须认真处理的边界。本文调用 RevKit Python `oracle_synth` 对 $n \leq 6$ 的 177 个传统函数逐一合成，全部返回；但返回的是 Rz-phase netlist，而不是可直接按 T/Tdg 完整计数的 Clifford+T netlist。角度审计显示 177 行中只有 6 行不含非 Clifford R_z ；171 行含非 Clifford R_z ，总数为 9242，角度除以 π 的最大分母为 64。因此，Resource-NMCTS 相对 RevKit lower-bound score 的 6/171/0 和平均 score 高 751.69% 不能解释为精确 Clifford+T T-count 失败，而应解释为不同 gate-set/phase 口径下的压力结果。

若对每个非 Clifford R_z 仅加入 1 个 score 单位，Resource-NMCTS 和 Pareto-Resource-NMCTS 分别转为 140/37/0 与 157/20/0；若加入 2 个 score 单位，二者均达到 177/0/0。进一步把 Resource-NMCTS、Pareto-Resource-NMCTS 和 FPRM polarity archive 作为 Resource-NMCTS family score-ranked portfolio 后， $\lambda_{R_z} = 1$ 时为 157/20/0， $\lambda_{R_z} = 1.5$ 时达到 177/0/0，而传统 baseline family 在 $\lambda_{R_z} = 1$ 时只有 80/97/0。若采用 Ross-Selinger 风格的 R_z 近似综合 proxy，按 $T/R_z = \lceil 3 \log_2(1/\epsilon) \rceil$ 计费，则在 $\epsilon = 10^{-3}$ 时 $T/R_z = 30$ ，Resource-NMCTS family 相对 RevKit synthesized-cost proxy 为 177/0/0，平均 score 降低 95.03%。这些结果仍是逻辑层成本模型，不是实际 rotation sequence 输出。

表 5: 外部 LUT proxy 与 RevKit phase/Rz 边界结果。

比较	指标	函数数	胜/负/平	平均 Δ
Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-83.77%
Pareto-Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-84.27%
ROS-style LUT proxy vs fixed $K = 4$ LUT	score	309	219/0/90	-18.12%
Resource-NMCTS vs RevKit <code>oracle_synth</code>	lower-bound score	177	6/171/0	+751.69%
Resource-NMCTS vs RevKit <code>oracle_synth</code>	score+1/Rz	177	140/37/0	-14.52%
Resource-NMCTS family vs RevKit	score+1/Rz	177	157/20/0	-17.89%
Resource-NMCTS family vs RevKit	score+1.5/Rz	177	177/0/0	-42.26%
Resource-NMCTS family vs RevKit	$T/R_z = 30$ proxy	177	177/0/0	-95.03%

7.3 高维函数级边界与 RevKit 超时探针

在 $n = 18$ 的 12 个随机 ANF 函数上, adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1, 平均 score 从 11349.06 降至 10670.94; 完整 Resource-NMCTS 平均 score 为 7315.62, 相对 adaptive screen 进一步降低 23.85%。这说明中等高维上 screen 是强候选生成器, 但不能完全替代 Resource tail。

在 $n = 20$ 边界函数上, FPRM root beam 和 fast linear-pair 均触发 300 s task timeout。Adaptive depth-2 screen 相对 single screen 获得 5/0/1, 平均 score 降低 7.47%; 集成该分支后, Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 11.80%。该切片中完整 Resource tail 与 adaptive screen 资源持平, 因此 screen-gated Resource-NMCTS 可把平均时间从 77.10 s 降至 20.71 s。这个结果支持运行时门控, 但也提示 $n = 20$ 切片的主要质量收益来自结构筛选而非深层 MCTS tail。

RevKit 高维探针则给出外部 API 的可扩展性边界。对 $n = 14$ 前 8 个函数逐行启动一次 RevKit `oracle_synth` 子进程, 并设置 30 s wall-time cutoff 后, 只有 1 行返回, 7 行触发 timeout。唯一返回函数 `anf_n14_10` 的 RevKit lower-bound score 为 2948.79, 而 Resource-NMCTS 普通 score 为 10358.47; 但该 RevKit netlist 含 32767 个非 Clifford R_z , 若仅加入 1/Rz 的符号成本, RevKit score 变为 35715.79, Resource-NMCTS 转为胜出。这个结果说明, 高维 RevKit API 不能在当前适配器下作为普通 paired benchmark 批量纳入均值; 即使返回结果, 其解释仍受 phase/Rz gate-set 口径支配。

表 6: 高维函数级边界与 RevKit 隔离超时探针。

切片	结果	解释边界
$n = 18$ 函数级	Resource-NMCTS 相对 adaptive depth-2 screen 平均 score -23.85%。	screen 有效, 但 Resource tail 仍提供额外收益。
$n = 20$ 函数级	Adaptive depth-2 screen 相对 AND-direct ANF 平均 score -11.80%; screen-gated Resource-NMCTS 时间从 77.10 s 降至 20.71 s。	主要收益来自结构筛选; FPRM root beam/fast linear-pair timeout。
$n = 14$ RevKit API	8 行中 1 行返回、7 行 30 s timeout; 唯一返回行含 32767 个非 Clifford R_z 。	高维 RevKit 是 adapter/scalability boundary, 不是资源均值比较。

7.4 结构级 AI 与高预算 frontier

Depth policy 在 $n = 14, 16, 18$ 项集上训练，并在 held-out $n = 20$ 项集上评估。Policy 相对 single screen 获得 42/0/6，平均 score 降低 6.57%；相对 all-depth adaptive 平均 score 只高 0.28%，但平均运行时间降低 34.71%。保守 depth-2 skip guard 在 held-out $n = 20$ test 中没有 false skip，96 个样本全部与 fixed depth-2 screen score 持平，并相对 all-depth adaptive 节省 24.74% 平均时间。

Depth-frontier policy 将高预算 depth-2/3/4 质量前沿学习化。在 $n = 20, 28, 40$ scale harness 中，它相对 fixed depth-2 获得 35/0/37，平均 score 降低 2.19%；相对 all-depth adaptive depth ≤ 4 平均 score 只高 0.97%，但节省 58.69% 时间。独立 seed 的 $n = 24, 28, 32, 40$ 泛化集进一步显示，frontier policy 相对 fixed depth-2 获得 40/0/56，平均 score 降低 1.85%。

扩大训练集后的 large frontier policy 在同一泛化集上相对 fixed depth-2 获得 56/0/40，平均 score 降低 2.34%；相对旧 frontier policy 为 17/0/79，平均 score 再降 0.49%；相对 all-depth adaptive 只高 0.10%，仍节省 53.50% 时间。Cost-aware frontier policy 则在相同泛化集上仍相对 fixed depth-2 获得 56/0/40，平均 score 降低 1.39%，但把相对 fixed depth-2 的 plan-time 增幅从 large policy 的 563.80% 降至 170.03%。相对 large policy，它以 +0.99% score 代价换取 33.02% 时间下降和 7.61% 辅助线 lifetime area 下降。因此，frontier policy 现在不只是单一模型，而是形成了“质量模式”和“快速质量模式”两个可选运行点。

表 7: 结构级策略与 depth-frontier 结果。

设置	比较	平均 Δ score	平均 Δ time
held-out $n = 20$	depth policy vs single screen, 42/0/6	-6.57%	+2328.66%
held-out $n = 20$	depth policy vs all-depth adaptive, 0/6/42	+0.28%	-34.71%
scale $n = 20, 28, 40$	frontier policy vs fixed depth-2, 35/0/37	-2.19%	+503.20%
scale $n = 20, 28, 40$	frontier policy vs all-depth, 0/16/56	+0.97%	-58.69%
large generalization	large frontier vs fixed depth-2, 56/0/40	-2.34%	+563.80%
large generalization	large frontier vs all-depth, 0/6/90	+0.10%	-53.50%
cost generalization	cost frontier vs fixed depth-2, 56/0/40	-1.39%	+170.03%
cost generalization	cost frontier vs large frontier, 1/48/47	+0.99%	-33.02%

7.5 验证桥接与 schedule proxy

项集级 scale 覆盖 $n = 20, 22, 24, 28, 32, 36, 40$ ，主 scale、extended scale、depth-frontier、frontier-policy rerun、独立泛化 run、large frontier policy 泛化 run 和 cost-aware frontier policy 泛化 run 合计 6600 个方法行。所有方法行均通过两层验证：factor plan 的 ANF 展开验证为 6600/6600，emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证为 6600/6600，mismatch 总数为 0。该结果说明大规模项集资源表不是只统计 plan，但仍需承认它不是完整 truth-table 枚举。

Truth-table bridge 进一步把完整验证推进到 $n = 21, 22, 23$ 。在原 bridge、large-policy $n = 23$ rerun 和 cost-aware $n = 23$ rerun 中，300/300 个方法行通过完整 truth-table oracle 验证、plan ANF 展开验证和 emitted-circuit GF(2) 符号验证。该 bridge 还复现了 frontier 信号：large frontier policy 在 $n = 23$ bridge 上相对旧 policy 为 1/0/5，平均 score 再降 0.48%，T-depth proxy 再降 0.45%；cost-aware frontier policy 则相对 large policy 以 +0.92% score 代价换取 56.29% plan time 下降和 12.62% lifetime area 下降。

表 8: 正确性验证与逻辑层 schedule proxy。

验证项或比较	结果	说明
项集级 plan ANF 验证	6600/6600	覆盖 $n = 20$ 到 40 的 scale/frontier/frontier-policy rerun 与泛化 run。
项集级 emitted-circuit 符号验证	6600/6600	GF(2) 多项式模拟通过, mismatch 总数为 0。
$n = 21, 22, 23$ 完整 truth-table oracle 验证	300/300	所有 emitted circuit 逐点验证通过。
large $n = 23$ frontier vs old frontier	score -0.48% ; T-depth proxy -0.45%	质量型 frontier 继续压缩旧模型 gap。
cost-aware $n = 23$ frontier vs large frontier	score $+0.92\%$; plan time -56.29% ; aux area -12.62%	快速质量型 frontier 用小 score 代价换取运行时间和 lifetime area。

8 讨论

本文最稳妥的贡献表述是“资源约束搜索”，不是“AI 全面战胜传统综合”。当前证据显示，ANF/FPRM 项集合提供了清晰的语义验证基础，Boolean-ring screen 提供了高维结构候选，MCTS 与神经先验提供了候选组合能力，depth policy 与 guard 控制搜索成本，depth-frontier policy 则把高预算质量前沿部分学习化。Large frontier policy 说明，只要提供更充分的 frontier teacher 数据，结构级 AI 的质量 gap 可以从 0.61% 量级压到 0.10% 量级；cost-aware frontier policy 则说明同一个策略框架可以通过标签目标移动到更快的质量折中点，而不是只能追求最低 score。

RevKit API baseline 是当前稿件中最重要的边界证据。在 Rz-phase lower-bound 口径下，RevKit 是一个强 baseline；但由于 171/177 行包含非 Clifford R_z ，这还不是可直接声称的精确 Clifford+T baseline。符号成本分析进一步说明，该缺口并非不可追赶：每个非 Clifford R_z 只计 1 个 score 单位时，Resource-NMCTS family 已在 157/177 行优于 RevKit，计 1.5 个单位时覆盖全部 177 行；传统 baseline family 在 1 个单位时只有 80/177 行优于 RevKit。近似 rotation synthesis proxy 则给出更强的边界：只要把非 Clifford R_z 近似到 Clifford+T，RevKit lower-bound 的优势就会迅速消失；但这个结论依赖模型常数和精度 ϵ ，不能替代真实 synthesis 输出。

本文必须明确四个限制。第一，SSHR 的 CNOT-oriented 优势仍存在；在 CNOT-only 或 CNOT-depth profile 下，本文优势会收窄。第二，RevKit API 的结果显示，本文当前 bit-flip emitter 并不支配 phase/Rz-rotation oracle synthesis；若论文不扩展到该口径，就必须把主张限定清楚。第三，高维 RevKit API 探针目前只是 timeout-boundary evidence，不是完整高维 RevKit baseline；返回行可以讨论 gate-set 口径，timeout 行不能进入资源均值。第四，schedule proxy 只是 emitted-circuit 层的逻辑调度代理，本文仍没有处理硬件 mapping、连通性、routing、magic-state factory 或噪声下可靠性。

下一步最有价值的提升目标不是继续堆内部表格，而是补两个外部缺口。其一，围绕 RevKit API

暴露出的强 Rz-phase lower-bound baseline，新增 phase/Rz-aware 的资源模型和 emitter，并显式处理非 Clifford rotation 的精确或近似综合成本；至少应把当前 proxy 替换为实际 gridsynth/Ross-Selinger 类 synthesis 输出的 T-count、T-depth 和 gate 序列审计，或者把本文题目收窄为 bit-flip oracle synthesis。其二，在 ROS-style LUT proxy 之外复现或接入官方 ROS、mockturtle 和 legacy RevKit/CirKit 流程，使比较从 proxy/API lower bound 扩展到更标准的外部流程。同时，当前 cost-aware 标签还应从单个时间惩罚扩展为显式多目标 teacher 或 Pareto policy，纳入 T-depth proxy 和辅助线 lifetime area。

表 9: 当前主张、证据与投稿边界。

主张	当前证据	边界
本文路线独立于 SSHR	状态、动作和验证都定义在 ANF/FPRM 项集合上；SSHR 只出现在 baseline 比较中。	不能声称 CNOT-only 支配 SSHR。
低 T-count 与低加权 score 是主优势	$n \leq 6$ 传统函数相对 ESOP、weighted ESOP-MILP、direct ANF 和 AND-direct ANF 有稳定优势。	资源权重仍是逻辑层人为设定。
Boolean-ring screen 是高维有效结构筛选	$n = 18, 20$ 函数级切片和 $n = 20-40$ 项集级 scale 均显示相对 single screen 的 score 改善。	高维不是全局最优性证明。
RevKit API 给出强外部边界	177 个传统函数全部返回；171/177 行含非 Clifford R_z ；Resource-NMCTS family 在 score+1.5/Rz 与 $T/R_z = 30$ proxy 下为 177/0/0。	RevKit lower-bound phase netlist 不是精确 Clifford+T 成本；高维 API 目前大量 timeout。
结构级 AI 能形成质量-时间折中	Large frontier policy 与 cost-aware frontier policy 均有泛化和 bridge 证据。	Large 是质量增强型；cost-aware 是快速质量折中型。
大规模结果具有语义可信度	6600/6600 项集级方法行通过 plan 和 emitted-circuit 符号验证；300/300 bridge 行通过完整 truth-table 验证。	$n = 24-40$ 仍未做完整 truth-table 枚举。

9 结论

本文提出 Resource-NMCTS，将量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题，并结合神经动作先验、MCTS、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 生成逻辑层可逆线路。实验表明，该方法在 $n \leq 6$ 传统函数上相对 ESOP/ANF/ROS-style LUT proxy 有显著低 T-count 与低加权 score 优势，在 $n = 18, 20$ 函数级边界上验证了 Boolean-ring screen 与 Resource tail 的互补作用，在 $n = 20$ 到 40 项集级 scale 中展示了结构级策略的可扩展性，并通过 $n = 21, 22, 23$ truth-table bridge 补强了高维正确性证据。

当前稿件已经形成独立于 SSHR 的投稿主线，但最终论文仍需围绕 RevKit/ROS/mockturtle 的外部工具链对齐、phase/Rz-aware emitter 和真正后端相关评估来提升审稿说服力。若这些补强完成，本文的主张就可以从“已有一条合理路线”推进为“有标准外部对照和清晰 gate-set 边界的资源约束 Boolean oracle synthesis 方法”。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.